

JAMAL CART

AI-PoweredE-Commerce Chatbot

AI Project Report

Group Member 1:	Shahana Mansoor Jamal
	B23110006152
Year:	3 rd year BSCS,Sec B
Department:	UBIT
University / Institute:	karachi University
Submission Date:	27 th july,2026

Intentionally blank

1. Abstract

JamalCart is an AI-powered conversational shopping assistant for a Pakistani e-commerce platform. The system combines a TensorFlow-based natural language intent classifier, rule-based entity extraction, and a strictly layered backend architecture (chatbot layer → service layer → database layer) to let customers browse products, manage carts and wishlists, place and track orders, apply coupons, and resolve common support issues entirely through natural-language conversation, alongside a conventional REST API and graphical frontend for the same functionality. The backend is built with FastAPI and PostgreSQL (via Supabase), and is exposed through both a custom web frontend and an evaluated Flutter mobile frontend.

2. Introduction & Problem Statement

Conventional e-commerce interfaces require users to navigate multiple menus, filters, and forms to accomplish tasks that could be expressed in a single natural-language sentence — e.g. “show me electronics under 5000” or “track my order”. This project addresses that gap by building a chatbot that sits in front of a complete e-commerce backend, translating free-text customer messages into concrete actions (searches, cart updates, order operations) while keeping the underlying system as maintainable and testable as a conventional API-driven application — rather than treating the chatbot as a thin, disconnected demo layer bolted onto a database.

A secondary problem addressed is graceful failure: most tutorial-grade chatbots either hallucinate an answer or return a dead-end “I don’t understand” message when a query falls outside their training data. This project instead routes unresolved or complaint-type queries (payment failures, damaged products, account lockouts, general dissatisfaction) to a human-escalation flow that notifies support staff by email, so the system degrades to a warm handoff rather than a failure.

3. Objectives

- Train a TensorFlow intent classifier on a custom-labeled dataset of 47 shopping/support intent categories.
- Extract structured entities (product, order ID, price, address, coupon code, quantity, city) from free text using rule-based regex NLP.
- Design and implement a strictly layered backend: routers → services → database, with zero SQL outside the service layer.
- Expose the complete e-commerce domain through a documented, testable REST API (FastAPI + OpenAPI/Swagger).
- Persist all data in a relational schema (PostgreSQL via Supabase) covering users, products, orders, payments, reviews, coupons, and chat logs.
- Build a working web frontend and evaluate a Flutter mobile frontend against the identical backend API.
- Implement multi-turn conversation state (e.g. asking a follow-up question and correctly resolving the reply) rather than treating every message independently.
- Implement a human-escalation path (email notification) for queries the model cannot confidently resolve.

4. Related Work

Rule-based and intent-classification chatbots (e.g. classic AIML-style or bag-of-words + feedforward network approaches) remain common in production customer-support systems because they are cheap to run, fully offline, and their behavior is auditable — in contrast to large language model (LLM) based assistants, which generalize far better to unseen phrasing but introduce cost, latency, and reduced predictability. This project deliberately uses the former approach per project requirements, while documenting the trade-off explicitly (Section 12) and outlining a migration path to embedding-based or LLM-based classification as future work.

5. System Architecture

The system follows a strict layered architecture, illustrated below:

User (web / Flutter / Swagger)

```
FastAPI routers (routers/*.py)      <- HTTP only, no business logic
```

```
chatbot/predict.py -> intent + confidence
```

```
chatbot/entity_extractor.py -> structured entities
```

chatbot/actions.py -> routes intent to a handler

```
state) | (+ chatbot/context_manager.py for multi-turn
```

services/*.py <- the ONLY layer allowed to query the database

```
database/supabase.py -> PostgreSQL(Supabase)
```

This separation means the exact same service functions back both the conversational chatbot flow and the conventional REST endpoints — e.g. checking out via chat and checking out via POST /checkout both call `services/order_service.py:checkout()`, so business rules are defined once.

6. Technology Stack

NLP / classification	TensorFlow (Keras), NLTK	Purpose
Entity extraction	Python re (regex)	Intent classification, tokenization, lemmatization
Backend API	FastAPI, Pydantic	Extract product/order/price/address/coupon entities REST endpoints, request/response validation
Database	PostgreSQL (Supabase), supabase.py	Persistent storage
Auth	bcrypt	Password hashing
Web frontend		Browser UI, served by FastAPI
Mobile frontend	HTML5, CSS3, vanilla JS	Cross-platform mobile UI, same REST API
Notifications		Escalation emails to support staff

Flutter / Dart

7. Database Design

The schema (seedatabase/schema.sql) consists of 11 relational tables:

Table	Purpose
users	Customer & admin accounts, hashed passwords, address/city
categories	Product categories (Men, Women, Kids, Electronics, ...)
products	Catalog: name, price, stock, brand, rating, SKU
cart	Per-user cart lines (user_id, product_id, quantity) Saved-for-
wishlist	later products per user
orders	Order header: status, payment method, total, address
order_items	Line items per order (product, qty, price at time of sale)
payments	Payment transactions per order
reviews	Product ratings/reviews per user
coupons	Discount codes and expiry dates

Table	Purpose
chatbot_logs	Every chat turn: question, predicted intent, response

8. Methodology — NLP Intent Classification

8.1 Dataset

A custom dataset of 48 intent categories was authored in chatbot/intents.json, each with 5–12 example phrasings (~290 labeled sentences total). Categories span product search/browsing, cart & checkout, order lifecycle (tracking, cancellation, refund), account management, and a customer-support complaint family (payment failure, damaged/wrong item, delayed order, warranty claim, account lockout, general complaint, feedback). See the accompanying dataset/DATASET.md for the full breakdown.

8.2 Preprocessing

- Tokenization via NLTK word_tokenize
- Lowercasing and lemmatization via NLTK WordNetLemmatizer
- Punctuation token removal
- Bag-of-words vectorization against a 330-token vocabulary built from all training patterns

8.3 Model Architecture

Input(330,) -> Dense(128, relu) -> Dropout(0.5) -> Dense(64, relu) -> Dropout(0.5) -> Dense(47, softmax)

Trained with categorical cross-entropy loss and stochastic gradient descent (learning rate 0.01, momentum 0.9, Nesterov, weight decay 1e-6) for 200 epochs, batch size 5. Training accuracy converges to approximately 93–98%.

8.4 Confidence Thresholding & Fallback

The intent 'fallback' has zero training patterns by design and is never a direct model output. Instead, routers/chat_router.py compares the model's top-prediction confidence against a threshold (CONFIDENCE_THRESHOLD = 0.70); below that threshold, the system substitutes 'fallback' regardless of what the model predicted. This keeps the decision to say “I don't know” an explicit, tunable business rule rather than something baked into the model's weights.

8.5 Entity Extraction

Entities (price, product ID, order ID, category, brand, city, address, quantity, coupon code) are extracted independently of the ML classifier, via regular expressions in chatbot/entity_extractor.py. This is a deliberate hybrid design: classification (what does the user want) is handled statistically; slot-filling (which product/order/address) is handled deterministically, which is both more accurate for structured values and avoids requiring a labeled entity dataset.

9. Implementation

9.1 Chatbot Layer

chatbot/actions.py implements a dictionary-based intent router (INTENT_HANDLERS), mapping each intent tag to a single handler function. Handlers never query the database directly — they call functions from services/. Multi-turn conversations (e.g. the bot asking “which product?” and correctly resolving the next message as the answer, rather than re-classifying it from scratch) are implemented via chatbot/context_manager.py, which tracks a small per-user state machine.

9.2 Service Layer

Each service module (product_service.py, cart_service.py, order_service.py, user_service.py, auth_service.py, chatbot_service.py, notification_service.py) owns exactly one part of the domain, validates its inputs, raises typed exceptions (ValidationError, NotFoundError, AuthError), and returns a consistent {success, message, data} envelope.

9.3 API Layer

FastAPI routers (routers/*.py) are deliberately thin: each route validates its Pydantic request model, calls exactly one service function via a shared run_service() helper (which translates the service's typed exceptions into the correct HTTP status code), and returns the result. This keeps routes free of business logic and try/except boilerplate.

9.4 Frontends

The web frontend (static/, templates/) is server-rendered HTML with vanilla JavaScript calling the same REST endpoints. A Flutter mobile frontend (flutter_frontend/) was additionally built against the identical API to demonstrate that the backend is genuinely client-agnostic — both frontends call the same /chat, /products, /cart, and /checkout endpoints with no backend changes required.

10. Testing

Endpoints were verified using FastAPI's TestClient against both a mocked Supabase client (to validate routing, status codes, and error handling in isolation) and the real trained model (to validate the full predict → extract → act → log pipeline end-to-end, including the confidence-threshold fallback path). [Insert your own manual test log / screenshots here.]

11. Results

[Insert screenshots here: Swagger UI endpoint list, a sample /chat request/response, the web frontend product grid and chat panel in use, and the Flutter app if built. Recommended: one screenshot per bullet in the 'Expected Output' section of README.md.]

12. Limitations

- The bag-of-words classifier has no notion of word order or semantics, so it generalizes weakly to phrasings not resembling its ~290 training examples — training accuracy (~93–98%) reflects fit to the training set, not real-world robustness.
- Product-name entity resolution relies on a capitalized-phrase heuristic and substring search rather than semantic matching, so free-text product references in lowercase or paraphrased form may fail to resolve.
- Conversation state (context_manager.py) is held in-memory per process; a multi-worker production deployment would need a shared store (e.g. Redis) instead.
- Coupon discount semantics are a flat PKR amount rather than a percentage, constrained by the existing coupons.discount column type in the live schema.

13. Future Work

- Replace or augment the bag-of-words classifier with sentence-embedding similarity (e.g. a small sentence-transformer computing cosine similarity against each intent's example patterns) for better generalization without requiring an LLM API dependency.

- Evaluate an LLM-based intent/entity extraction pipeline as a higher-accuracy alternative, with cost/latency trade-offs documented.
- Move conversation state to a shared store for multi-worker deployment.
- Add a proper loyalty/membership program and real percentage-based coupon support.
- Expand the Flutter frontend to full feature parity with the web frontend (wishlist, order history, coupon UI).

14. Conclusion

This project demonstrates that a conversational interface can be layered cleanly on top of a conventional, well-architected e-commerce backend rather than being built as an isolated demo. The TensorFlow intent classifier, while limited by dataset size as documented above, successfully routes 47 distinct intent types through a shared service layer that also powers a full REST API and two independent frontends. The explicit documentation of the classifier's limitations, and the concrete, architecture-compatible upgrade paths identified in Section 13, are presented as part of the project's engineering contribution rather than as unaddressed shortcomings.

15. References

- FastAPI documentation — <https://fastapi.tiangolo.com/>
- TensorFlow / Keras documentation — <https://www.tensorflow.org/>
- NLTK documentation — <https://www.nltk.org/>
- Supabase documentation — <https://supabase.com/docs>
- Flutter documentation — <https://docs.flutter.dev/>
- [Add any papers, tutorials, or course material you referenced.]

Copyright © 2026 Shahana Mansoor Jamal.

All Rights Reserved.

Jamal Cart and all associated source code, documentation, software architecture, database design, AI models, and project materials are the intellectual property of Shahana Mansoor Jamal.

Unauthorized copying, distribution, modification, publication, or commercial use of this work without prior written permission is prohibited.